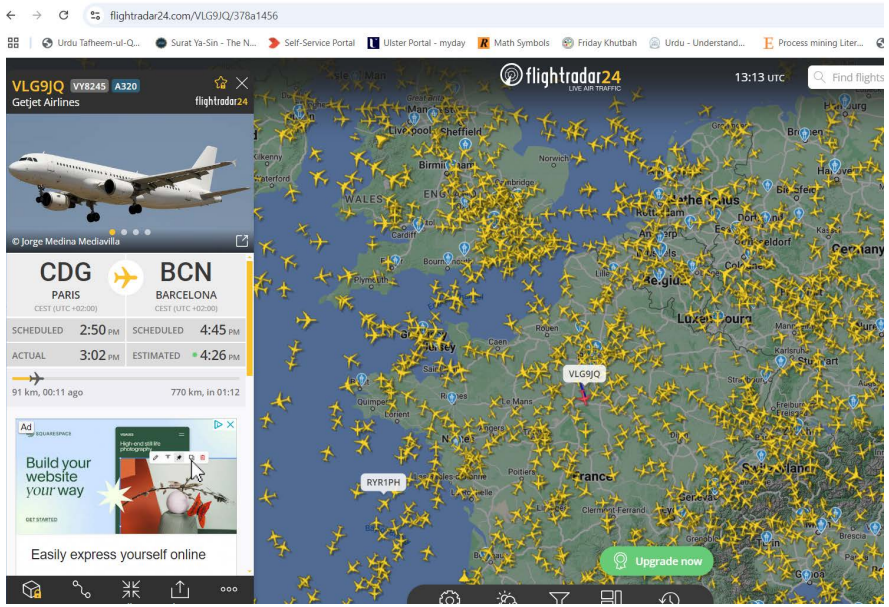


Travel Journal Platform

A platform for users to create travel journals and share photos, videos and audio recordings from their travels.

This is a sample CW, for the purpose of reference only.
Alot of details are removed due to data protection. This CW is based on old specs of the module.

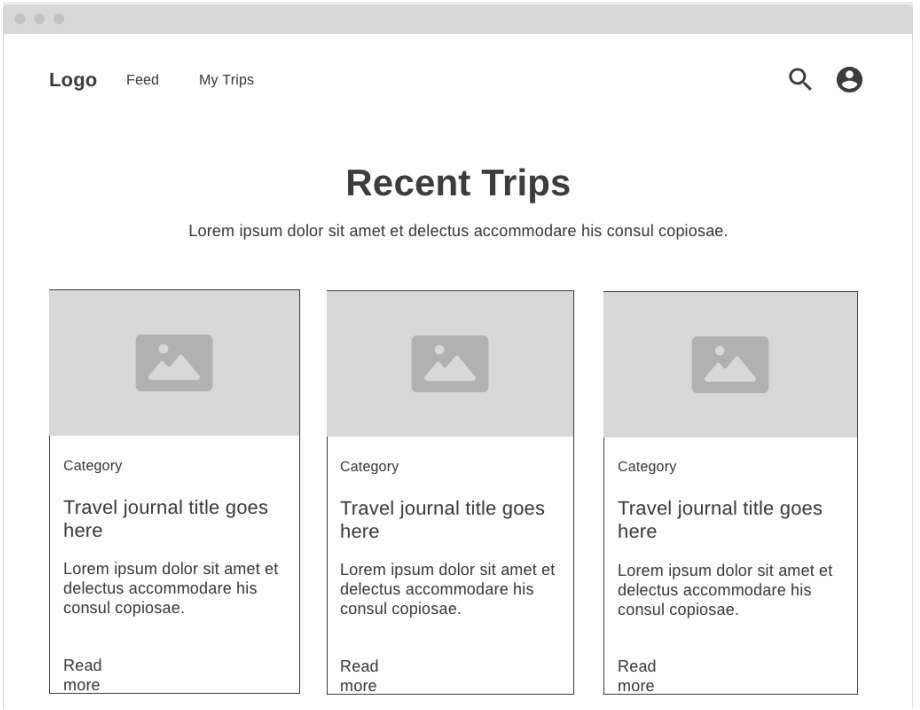
Student Name - B00000000

Problem Definition

Travel Journal Platform is a multimedia, social media site that will allow users to register and share their trips with other users on the platform. Users will be able to create resources called ‘journals’ in which they can write about their trip, and upload supporting media ranging from photos, videos, and audio recordings. Each type of media will have additional metadata. The platform essentially acts as a multi-tenant Software-as-a-Service (SaaS). Users access the site through a client without the need for prior software installation, and the platform offers media sharing and editing tools.

This immediately presents a variety of issues pertaining to scalability:

- User load is .....
- Database performance .....
- As this is a media sharing site, media storage .....



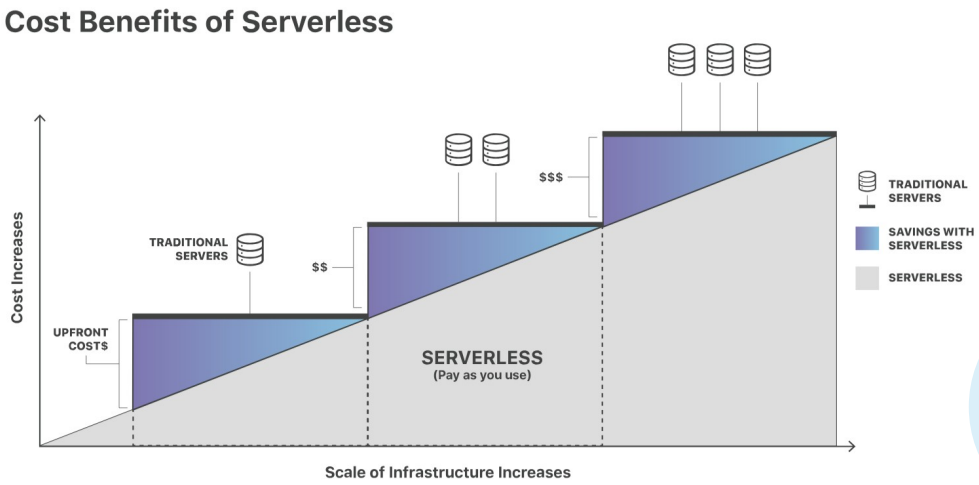
This is a sample CW, for the purpose of reference only.
Alot of details are removed due to data protection. This CW is based on old specs of the module.

Scalability and the appropriateness of using cloud architecture

The domain is well-suited to using cloud architecture due to several factors. Firstly, the application can be used by a varying number of clients at any given time. Cloud infrastructure allows for dynamic scaling as the number of users fluctuates. Consequently, the cost is reduced as the payment model is based on services used, as opposed to upfront payment for on-premises equipment, which is costly (Cloudflare, 2024).

Furthermore, there is an immense quantity of media that can be uploaded by many users. Cloud infrastructure provides autoscaling with both horizontal and vertical scalability (Microsoft Azure, 2024, Scaling) for storage and other services that on-premises infrastructure cannot compete with. This ensures the platform can handle the exponential growth of media content without performance degradation.

From a developer's perspective, the cloud provides me .....



Credit: CloudFlare

Chosen cloud model

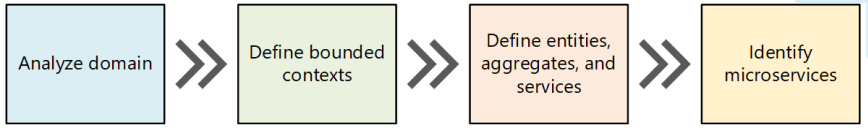
The domain will follow the microservices cloud model. By using this model, the domain can be split into individual services which are isolated from one another, and which are responsible for their own data persistence. The advantages are ....

- Better fault isolation – if one service fails, the others aren't affected in the same way.
- Data isolation – Schema updates are easier because they only affect a single service.
- Supports polyglot programming – each service can use technologies independent of the others, and which are more suited for the task at hand.
- Deployed/scaled independently – each service can be deployed and scaled using orchestration services or serverless methods.

I investigated using other cloud models, such as N-tier, however I wanted to avoid a monolithic design and have a clear separation between each service. Event-driven architecture was also a suitable alternative, and I have in fact used some elements of this model within my own domain with event driven communication between services.

Using a microservices approach with event-driven communication provides .....

Domain Driven Analysis

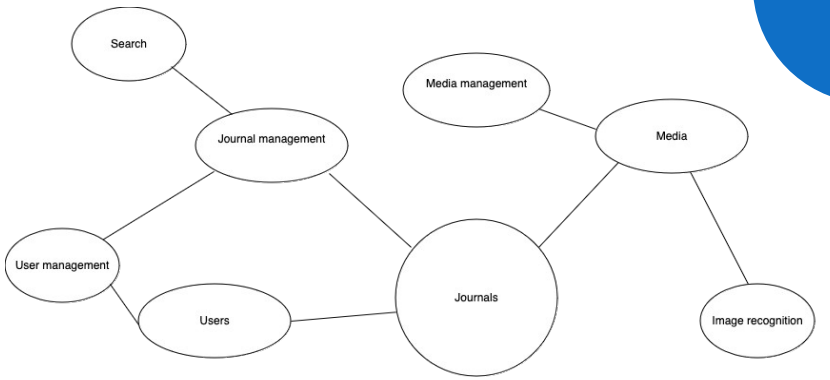


Credit: Microsoft Azure

The process for designing a microservices architecture differs greatly from other approaches which can lead to some common design problems (EdPrice-MSFT, n.d., Domain Analysis). As such, Azure suggests the following steps:

1. Domain Analysis to model microservices
2. Use Domain Driven Design (DDD) to design microservices
3. Identify microservice boundaries.

Using DDD .....



Simple bounded context diagram showing the domain's primary components

Identifying entities, data structures and microservices

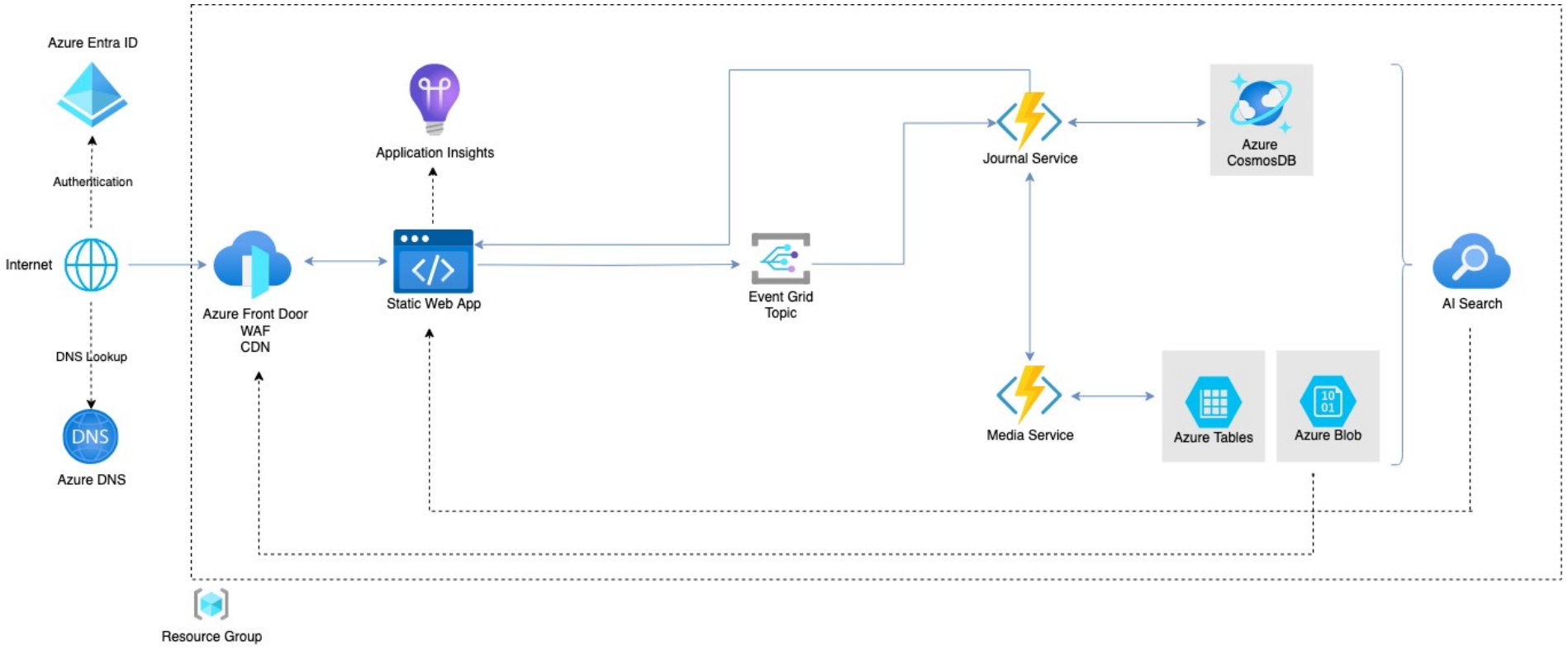
With the help of my domain analysis, I identified the primary entities to be stored. **Users**, **journals, media** and metadata. I have selected what I believe to be the appropriate data structure and storage for my application.

- **Users** – U.....
- **Journals** – I.....
- **Media** – Media.....

This is a sample CW, for the purpose of reference only.
Alot of details are removed due to data protection. This CW is based on old specs of the module.

With these entities in mind, the identified microservices are the following: journal service, media service and a user service via Azure Entra ID. Each with their own persistent storage. These will serve data back to a static webpage.

Architecture diagram



Architecture explanation

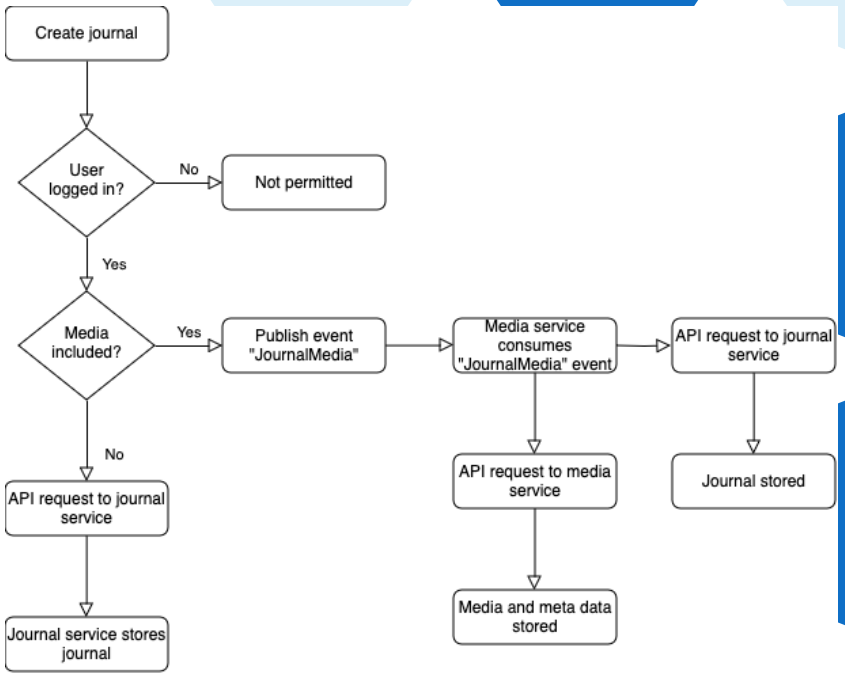
Design Patterns

Several design patterns have been adhered to when designing the domain:

- Publisher/Subscriber pattern for messaging...
- Static Content Hosting pattern for delivering content to client....
- Backends for Frontends pattern separating backend services from frontend....

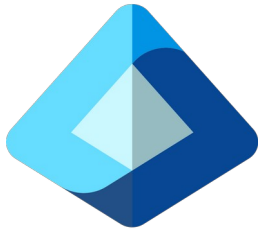
General Overview

- The entry-point to the domain is Azure Front Door, acting as a CDN and a firewall. With Azure DNS for advanced DNS capabilities.
- Azure Entra ID is the service chosen to ....
- The front-end of the application is hosted on an Azure Static Web App which ....
- The journal and media services are REST API Azure Function apps which handle ....
- Each of the services have their own persistent storage. Azure CosmosDB using MongoDB for ....
- The Azure Function services communicate via event-driven communication with Azure Event Grid. For example, ...



Example control flow diagram for the creation of a new journal might appear.

Selected Azure services



Azure Entra ID has been chosen for its wide range of functionality. It helps with streamlining user authentication and identity management, including features such as single sign-on and multifactor authentication. It also has a range of APIs for managing accounts and there is support for open protocols such as OAuth 2.0 and OpenID (rolyon, n.d). Some downsides to Azure Entra ID .... An alternative I considered was creating .....

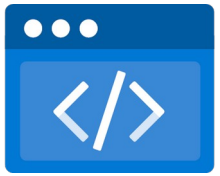
Azure Front Door is the entry-point to the application, a ....





Logo credits: Microsoft Azure

Selected Azure services cont.



Azure Static Web Apps has been chosen to host the static front-end. It is designed for g....

Azure Event Grid provides for the event-driven communication between services. It is a fully managed publisher-subscriber messaging service for building event-driven services and is designed for high throughput and reliability (Spelluru, 2024). For example, ...



The back-end REST API services are comprised of Azure Function apps. By leveraging Functions, I can focus on solving problems without having to concern myself with the underlying infrastructure involved. There are a wide choice of languages to choose from when using Functions and it comes with integrated security to protect HTTP-triggered functions. Furthermore, Functions is serverless, meaning that it scales appropriately based on demand and only bills based on usage (Microsoft Azure, n.d., Azure Functions). Although Functions is suited to building APIs and by extension microservices, there are certainly alternatives that are better suited to larger scalability. Options such as ....



Logo credits: Microsoft Azure

Selected Azure services cont.



As detailed previously, journals are not intended to be consistent across entries, and so a NoSQL solution is a strong choice. To this end, I have identified Azure CosmosDB using MongoDB as the appropriate choice for storing journal data. It has desirable features such as being serverless, having auto and instant scaling based on demand and is a fully managed service (seesharprun, n.d.). Additionally, document retrieval with MongoDB and CosmosDB is in single-digit milliseconds range, and because CosmosDB is a multi-model DB, I'll be able to leverage Azure's multiple APIs.



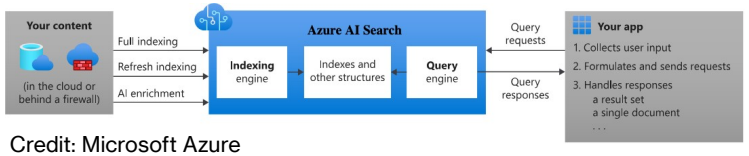
Azure Blob is scalable object store for storing large amounts of unstructured data, perfect for storing my application's media. It will aid in providing my application scalability as I won't have to worry about the infrastructure for having to store enormous number of media. Moreover, it is built for high-availability with local, zone and geo-redundant storage (Microsoft, 2022). It also operates o....



Azure Table Storage has been chosen to store the application's media metadata. It is a service that stores non-relational structured metadata. It's a schemeless key-value store, and each entry can be indexed and associated with a particular media. A benefit to using Azure Table Storage is that when querying data, t.....

Logo credits: Microsoft Azure

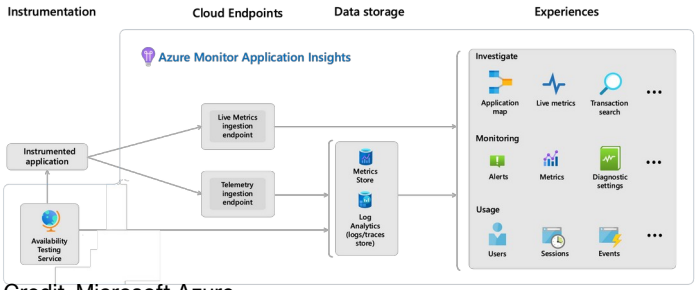
Advanced Features



Credit: Microsoft Azure







Credit: Microsoft Azure

I have decided on implementing Azure AI Search and Azure Application Insights as my advanced features. AI search will improve user experience while Application Insights will make maintaining the application easier.

Azure AI Search ....

Application Insights provides a dashboard on the Azure portal for ....

Assessment of limitations

There are some key limitations with this domain:

1. Firstly, regarding scalability, I have designed this domain using ...
2. With microservices, I am introducing more moving parts when compared with a monolithic domain. Despite having simpler and isolated services w....
3. Data integrity is a concern due to each service being responsible for its own persistence. An effective remedy t...
4. The complexity of the architecture can lead to ...
5. Specific services have their own particular limitations. Azure Static Web Apps can have cold start issues, sometimes ...

Evaluation of scalability

Strengths:

- Serverless architecture with Azure Static Web Apps and Azure Functions provides my domain with a serverless environment, allowing the application to scale automatically based on demand without a need for manual intervention.
- Microservices model promotes independent scaling. I can scale each service independently based on its load, enhancing overall domain scalability.
- Event-driven communication enables decoupling and asynchronous communication between services. I can then scale each service to handle the optimal load.
- My storage services, Azure CosmosDB and Azure Blob Storage are designed to handle large volumes of data and can scale automatically to meet demand.



Weaknesses:

- A weakness of my microservices approach is the management and coordination of multiple services. This complexity in services introduces ....
- The distributed nature of my approach can result in latency, particularly if I.....
- As previously mentioned, the services built on Azure Functions are limited by the the size of my Functions plan. For greater scaling, other Azure services are required such as .....

In summary, my domain leverages serverless architecture and microservices to achieve high scalability, but I'm limited by overall application complexity, and potential issues with latency.

Conclusion

I believe that I have built an effective, scalable and highly-available microservices domain for the task at hand. By utilizing Static Web Apps and Functions I have built ...

Event-driven communication, implemented using Azure Event Grid further improves the domain workflow by enabling asynchronous communication between services and improving coordination. Additionally, I have ...

However, there are limitations to consider, such as ....

In summary, the domain is designed to be robust, flexible and capable of meeting its dynamic needs. By acknowledging the limitations imposed by my domain architecture and providing potential solutions, I have ensured that the domain is well-prepared for future challenges.

Bibliography

- Aaron Maxwell (n.d.). *Application Insights overview - Azure Monitor*. [online] learn.microsoft.com. Available at: https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview.
- Cloudflare (n.d.). What Is Serverless Computing? | Serverless Definition | Cloudflare UK. *Cloudflare*. [online] Available at: https://www.cloudflare.com/en-gb/learning/serverless/what-is-serverless/.
- duongau (n.d.). *Azure Front Door*. [online] learn.microsoft.com. Available at: https://learn.microsoft.com/en-us/azure/frontdoor/front-door-overview.
- .
- .
- .
- .
- .
- .
- .